

JPEG-Like Image Compression

Yuvraj Singh (210050172)
Arnav Aditya Singh (210050018)

November 24, 2024

Contents

1	Introduction	2
2	Problem Statement	2
3	Algorithms	2
3.1	Basic Implementation	2
3.2	Advanced Implementation: Anisotropic Diffusion	3
3.2.1	Basic Idea of Edge-Enhanced Diffusion	3
3.2.2	Compression	4
3.2.3	Implementation	4
4	Datasets	4
5	Plots	5
5.1	RMSE vs. BPP for 20 different images	5
5.2	Mean Compression Ratios for different datasets	9
5.3	Comparing Different Quant Matrices	9
6	Results	10
6.1	Basic Implementation	10
6.2	Comparison with Existing Implementations	10
7	Contributions	11
8	References	11

1 Introduction

Image compression is a crucial area in digital image processing, enabling efficient storage and transmission of image data by reducing file sizes while maintaining acceptable visual quality. This project focuses on implementing a JPEG-like image compression system, exploring both grayscale and color images, and comparing its performance with other state-of-the-art methods.

2 Problem Statement

The goal of this project is to design and implement a JPEG-like image compression system with the following components:

- Computation of 2D Discrete Cosine Transform (DCT) coefficients of non-overlapping image patches.
- Implementation of the quantization step to reduce precision while preserving critical details.
- Construction and usage of Huffman trees for entropy encoding.
- Writing the compressed data to a file in a specific format, with functionality to read and decompress the data accurately.
- Simulation of different quality factors, plotting RMSE (Root Mean Square Error) versus BPP (Bits Per Pixel) for at least 20 images.

Additionally, the project explores:

- Comparison with an existing JPEG implementation (e.g., MATLAB or GIMP) under varying parameter settings.
- Experiments on color images, including chrominance subsampling in the YCbCr color space.
- Implementation of a research paper: *C. Schmaltz, J. Weickert, and A. Bruhn, "Beating the Quality of JPEG 2000 with Anisotropic Diffusion," DAGM 2009.*

3 Algorithms

3.1 Basic Implementation

The JPEG-like compression system consists of the following steps:

1. **2D DCT Computation:** Non-overlapping 8x8 patches of the image are transformed using the 2D DCT to achieve energy compaction.
2. **Quantization:** The transformed coefficients are divided by a quantization matrix and rounded to integers. Different quality factors are simulated by scaling the matrix.

3. **Huffman Encoding:** The quantized coefficients are stored using Run Length Encoding followed by Huffman encoding.
4. **File I/O:** The compressed data is written into a binary file (e.g., using Python's `pickle`). The decompression process involves reading, decoding, dequantizing, and applying the 2D Inverse DCT (IDCT).
5. **Color Images:** For color images, RGB input is converted to YCbCr. The chrominance channels (Cb, Cr) are downsampled, compressed, and then upsampled during decompression.

3.2 Advanced Implementation: Anisotropic Diffusion

The research paper by Schmaltz et al. introduces an advanced image compression method using anisotropic diffusion. This method surpasses JPEG 2000 in terms of quality for certain scenarios.

3.2.1 Basic Idea of Edge-Enhanced Diffusion

The rationale behind diffusion-based image compression is the assumption that for small patches of images, the interior values of the patch are strongly correlated with the values at the boundary. Thus if we can acceptably reconstruct the patch from the boundary, we can store the image in a much smaller space.

Conventionally, diffusion is performed using the following equation

$$\partial_t u = \nabla^2 u = \nabla \cdot \nabla u$$

where ∇^2 is the Laplacian operator and $u(.,., 0)$ is the initial image (here containing only the boundary values). However, due to its averaging nature, this operator is bound to lose information of edges present within the patch.

To fix this, the authors propose using the following operator instead

$$Lu = \nabla \cdot (D \nabla u)$$

That is, the inner gradient is replaced with the gradient acted upon by a pixel-specific 2×2 linear operator D . The desired property of D should be that it enhances diffusion parallel to edges and suppresses diffusion across edges, i.e. it enhances diffusion perpendicular to ∇u and suppresses diffusion parallel to ∇u . So the eigenvectors of D should be $v_1 \parallel \nabla u$ and $v_2 \perp \nabla u$ with corresponding eigenvalues $\lambda_1 = g(|\nabla u|^2)$ and $\lambda_2 = 1$. Here g is a function controlling how strong this effect should be. The larger the value of g , the weaker this effect, i.e.,

$$g(|\nabla u|^2) = \frac{1}{1 + \sqrt{\frac{|\nabla u|^2}{l^2}}}$$

where l is the *contrast parameter*. The paper proposes using

$$D = g(\nabla u_\sigma \nabla u_\sigma^\top)$$

where ∇u_σ is the gradient of $K_\sigma * u$.

3.2.2 Compression

The paper’s novel contribution is the tree-like pattern for compression. We try to reconstruct the image by performing diffusion from a small number of edge pixels. If the reconstruction was satisfactory, we simply store these edge values. Else, the image is split along the longer axis and each half is recursively stored thus.

3.2.3 Implementation

We attempted to implement the above in `anisotropic_diffusion.py`. Unfortunately we were not able to get the above diffusion step to converge satisfactorily.

4 Datasets

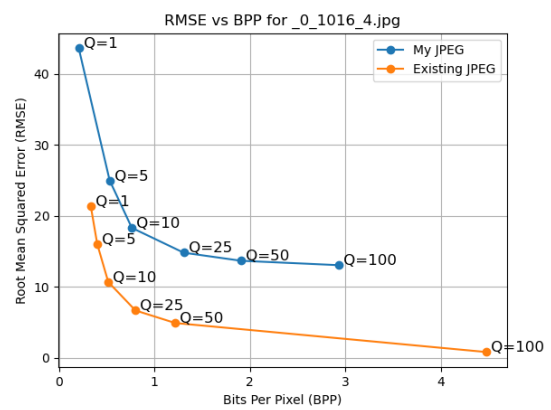
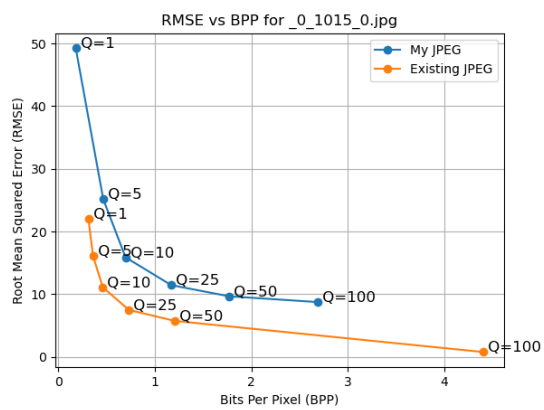
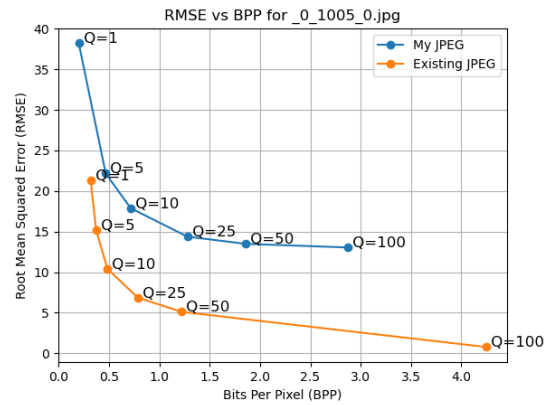
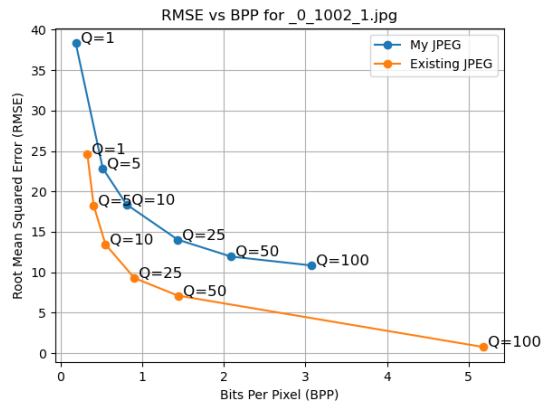
The algorithms were tested on a variety of datasets:

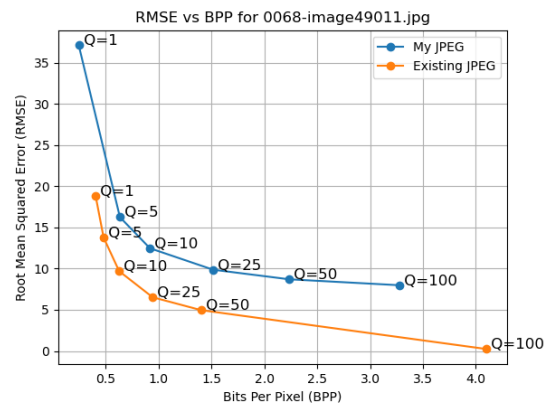
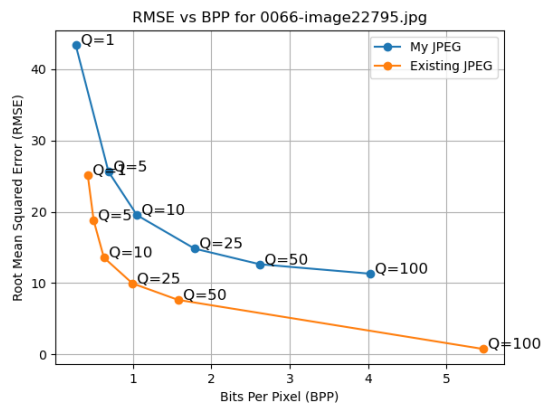
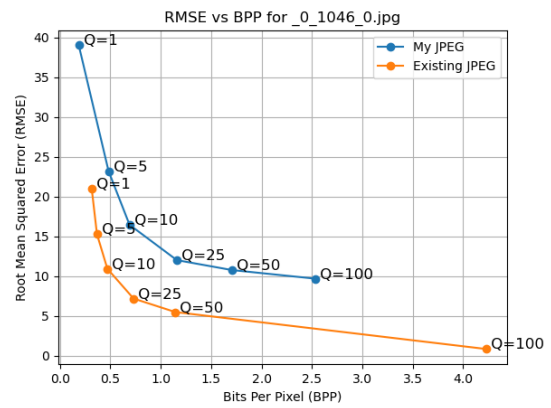
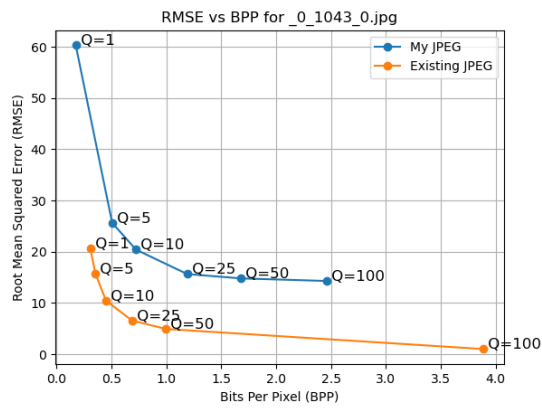
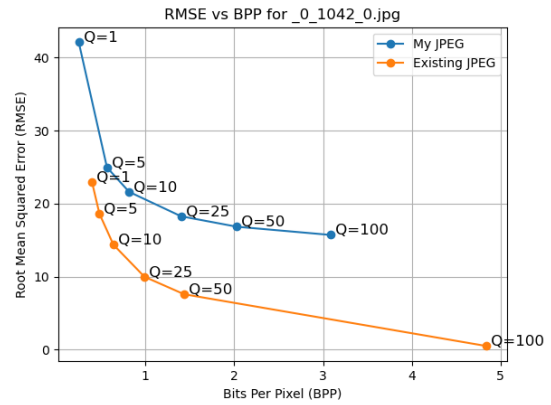
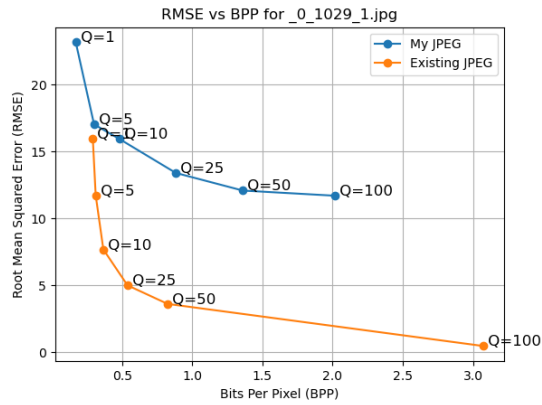
- Facial Landmark Detection by Deep Multi-task Learning
- COIL-100
- Airplane and Flamingo from Caltech-101
- CMU Dataset - AirFreight

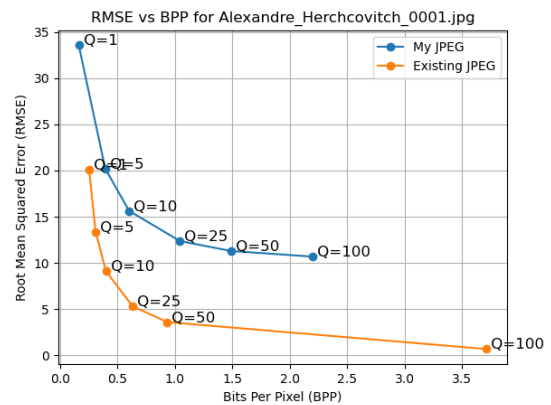
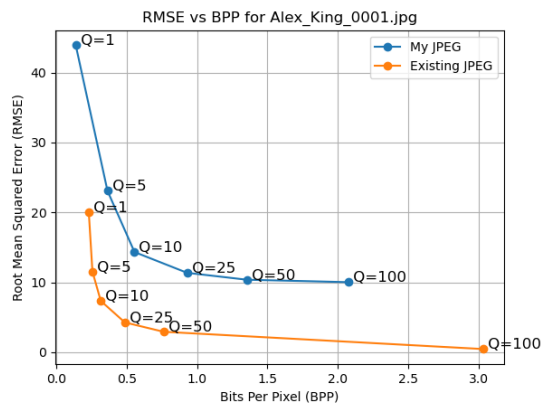
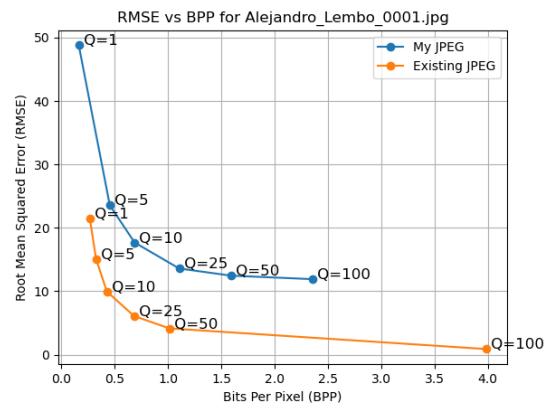
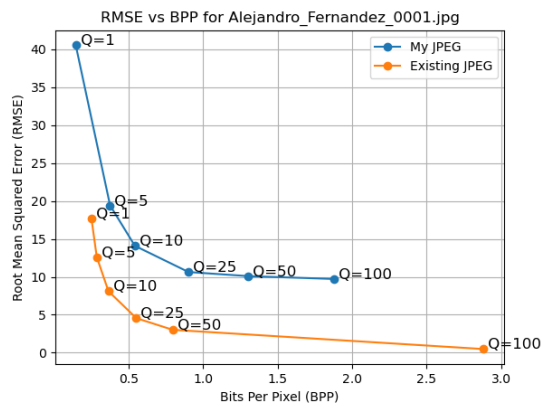
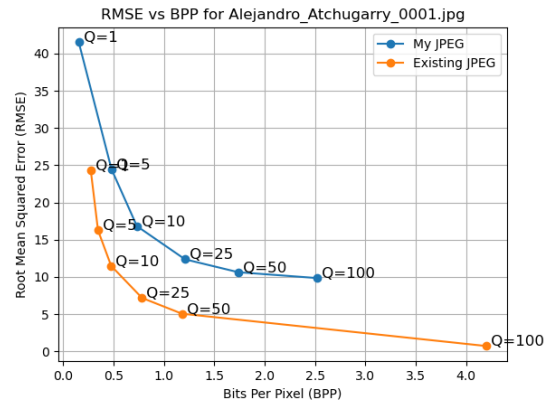
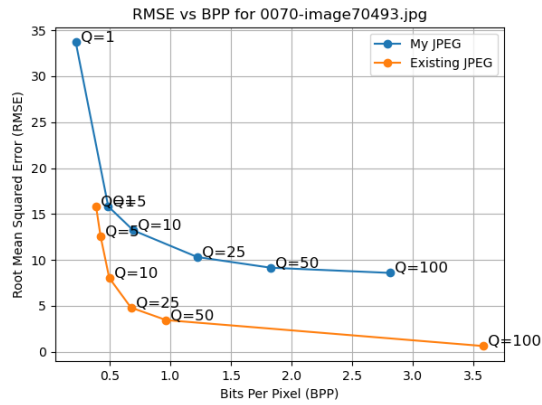
A total of 20 images were used to plot RMSE vs. BPP curves for different quality factors from the first dataset. Mean Compression Ratios for the datasets are calculated using 50 images from each dataset.

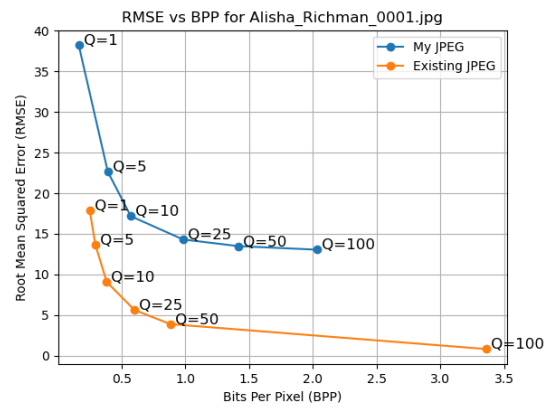
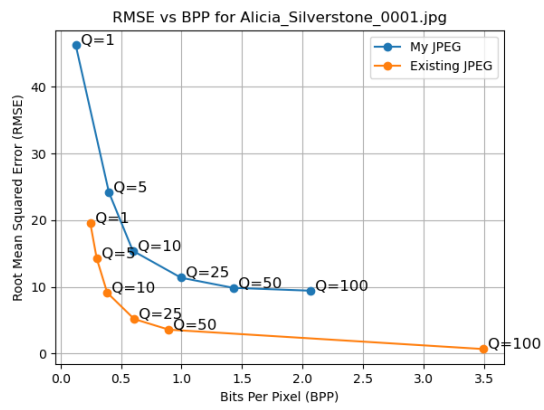
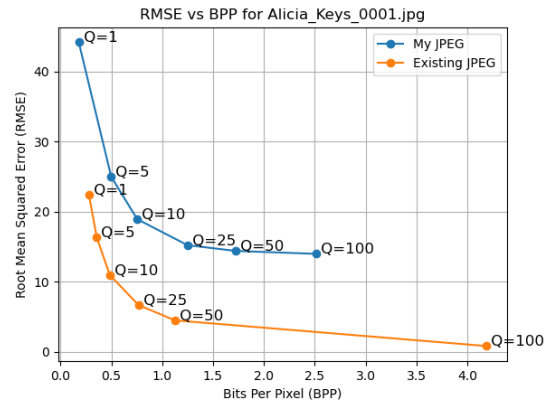
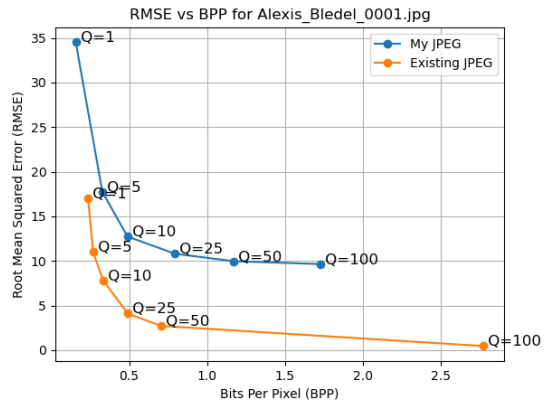
5 Plots

5.1 RMSE vs. BPP for 20 different images

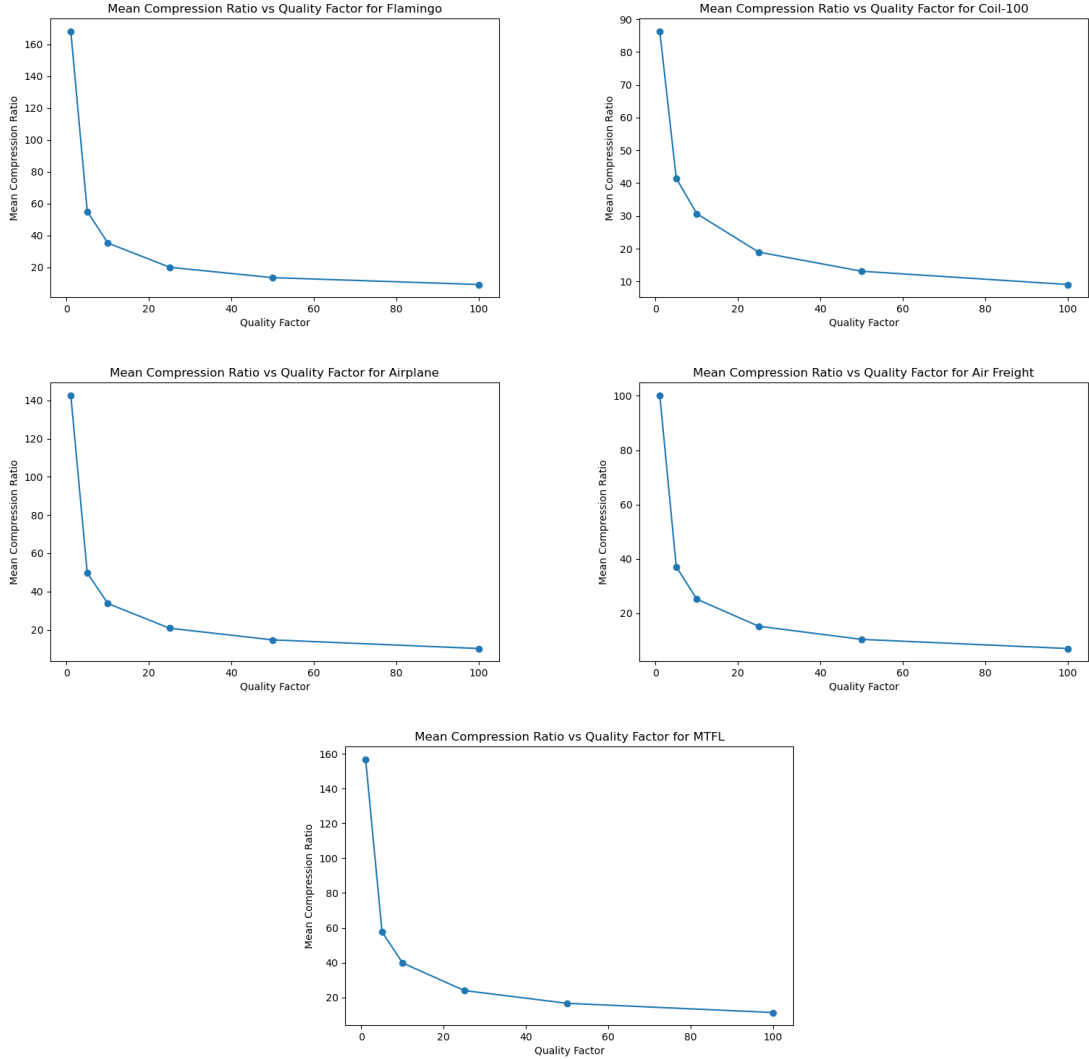








5.2 Mean Compression Ratios for different datasets



5.3 Comparing Different Quant Matrices

```
qm1 = np.array([ [16, 11, 10, 16, 24, 40, 51, 61], [12, 12, 14, 19, 26, 58,
60, 55], [14, 13, 16, 24, 40, 57, 69, 56], [14, 17, 22, 29, 51, 87, 80, 62],
[18, 22, 37, 56, 68, 109, 103, 77], [24, 35, 55, 64, 81, 104, 113, 92], [49,
64, 78, 87, 103, 121, 120, 101], [72, 92, 95, 98, 112, 100, 103, 99] ])

qm2 = np.array([ [17, 18, 24, 47, 99, 99, 99, 99], [18, 21, 26, 66, 99,
99, 99, 99], [24, 26, 56, 99, 99, 99, 99, 99], [47, 66, 99, 99, 99, 99, 99,
99], [47, 66, 99, 99, 99, 99, 99, 99], [99, 99, 99, 99, 99, 99, 99, 99], [99,
99, 99, 99, 99, 99, 99, 99], [99, 99, 99, 99, 99, 99, 99, 99] ])

qmv1 = np.array([ [16, 14, 13, 15, 19, 28, 37, 55], [14, 13, 15, 19, 28,
37, 55, 64], [13, 15, 19, 28, 37, 55, 64, 83], [15, 19, 28, 37, 55, 64, 83,
103], [19, 28, 37, 55, 64, 83, 103, 117], [28, 37, 55, 64, 83, 103, 117, 117],
[37, 55, 64, 83, 103, 117, 117, 111], [55, 64, 83, 103, 117, 117, 111, 90]
])

qmv2 = np.array([ [18, 18, 23, 34, 45, 61, 71, 92], [18, 23, 34, 45, 61,
71, 92, 92], [23, 34, 45, 61, 71, 92, 92, 104], [34, 45, 61, 71, 92, 92, 104,
```

115], [45, 61, 71, 92, 92, 104, 115, 119], [61, 71, 92, 92, 104, 115, 119, 112], [71, 92, 92, 104, 115, 119, 112, 106], [92, 92, 104, 115, 119, 112, 106, 100]])

qm1, qm2 are widely used quantisation matrices for the luminance and chrominance channels. We also considered the matrices qmv1, qmvr as suggested by [2].

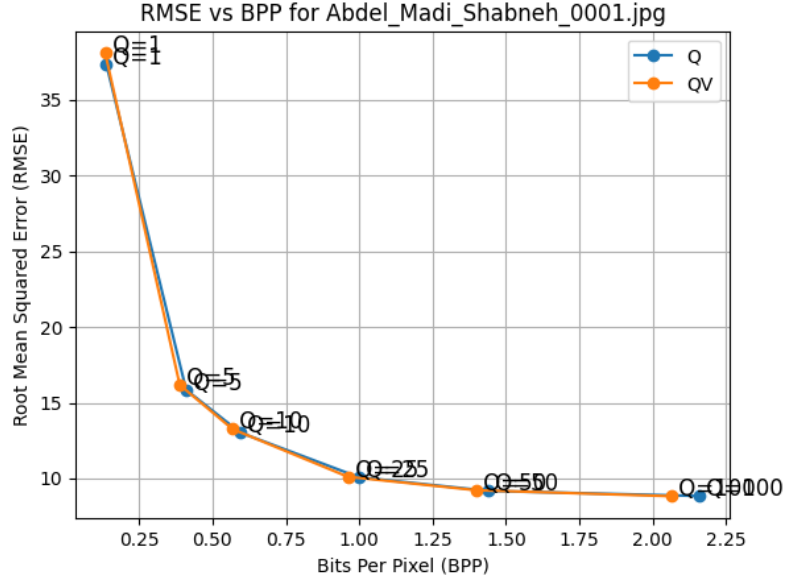


Figure 6: Comparing qm1,qm2(Q) with qmv1,qmvr(QV)

6 Results

6.1 Basic Implementation

The following results were observed:

- The RMSE vs. BPP curves demonstrate that increasing quality factors result in lower RMSE at the cost of higher BPP.
- Chrominance subsampling in color images achieved significant compression without noticeable visual degradation.
- Mean Compression ratios are not strongly dependent on the class of images used

6.2 Comparison with Existing Implementations

Experiments were conducted to compare:

- Different quantization matrices.
- Results with Python cv2's built-in JPEG compression.

7 Contributions

- **Yuvraj Singh:** Implemented the basic JPEG-like compression system and extended it to color images.
- **Arnav Aditya Singh:** Implemented the anisotropic diffusion-based compression method and run length encoding in base implementation.

8 References

1. C. Schmaltz, J. Weickert, and A. Bruhn "Beating the Quality of JPEG 2000 with Anisotropic Diffusion," *DAGM*, 2009.
2. F. Ernawan, and S. Nugraini "The Optimal Quantization Matrices For JPEG Image Compression From Psychovisual Threshold"